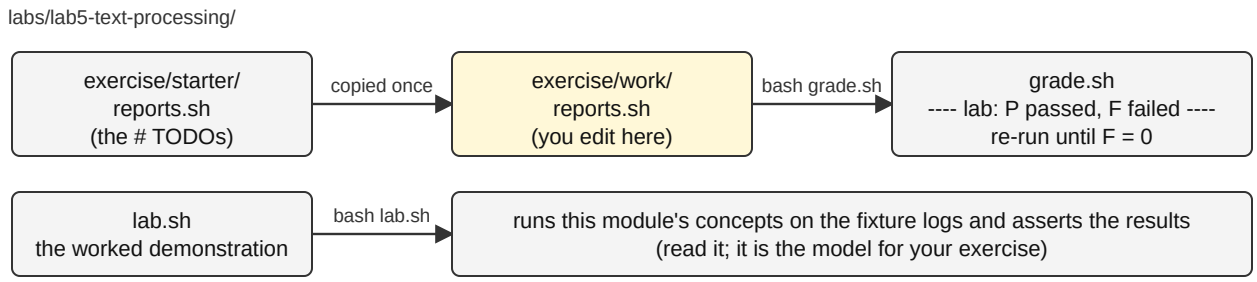
Lab 5 — The Text-Processing Toolkit: Two Reports as Pipelines (Instructor Version)

Instructor Note: Gray boxes and answers marked in red appear in the instructor copy only.

Topology



Environment Table

File	Where	What it is	You edit it?
`labs/lab5-text-processing/exercise/TASK.md`	course folder	The short statement of the exercise	No
`labs/lab5-text-processing/exercise/starter/reports.sh`	course folder	Two functions with `# TODO` comments	No — the grader copies it for you
`labs/lab5-text-processing/exercise/work/reports.sh`	created by `grade.sh`	**Your** copy of the starter	**Yes**
`labs/lab5-text-processing/exercise/grade.sh`	course folder	The grader: 5 checks	No
`labs/lab5-text-processing/exercise/work/REFLECTION.md`	you create it	Your two reflection answers	**Yes**
`seed/make_logs.sh`	course folder	Generates the fixture logs the grader uses (same output every run)	No

Objectives

- **Part 1:** Prepare the exercise folder and read the grader's first report.
- **Part 2:** Build `top_ips` — the "top N talkers" pipeline.
- **Part 3:** Build `count_class` — an `awk` filter that counts one class of status codes.
- **Part 4:** Verify with the grader until every check passes.
- **Part 5:** Write the two reflection answers.

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.

Background / Scenario

`logsleuth`, the log-analysis tool you are building through this course, has two reports that are pure shell pipelines: **`top`** (which client IPs sent the most requests) and **`summary`** (how many responses were 2xx, 4xx, 5xx). In this lab you write both reports as functions in `reports.sh`.

The data is the fraud-clf API gateway log from the SecMLOps program — the same log format you worked with in module M05. Each line is one request; **field 1** is the client IP and **field 9** is the HTTP status code:

```
10.0.0.1 - - [16/Oct/2026:13:00:00 -0500] "GET /api/health HTTP/1.1" 200 512 "-" "kube-probe/1.31"
```

The grader regenerates the fixture log on every run with `seed/make_logs.sh`, so the numbers never change: the top IP is `10.0.0.1` with **50** requests; there are **85** 2xx responses, **50** 4xx and **5** 5xx. Those are the numbers your functions must produce.

Note: Everything in this lab runs offline inside the course container (or any shell with bash 5 and GNU `awk`, `sort`, `uniq`). Nothing needs Docker beyond the container itself.

Required Resources

- The course folder, opened in a lab session (`docker run --rm -it -v "$PWD":/course -w /course bash-course:2.0`), or a shell where `bash tools-check.sh` reports `ready`.
- Module M05 (steps S05.3 and S05.4 are the ones this lab applies).
- A text editor. You may consult documentation and use AI assistance for lookups; say so in your reflection. Do **not** open `solution/`.

> **Instructor Note:** Budget 55 minutes of learner time; grading takes about 5 minutes per learner. The starter fails all five checks (observed: `---- lab5: 0 passed, 5 failed ----`); the reference solution passes 5/5. The most common wrong turn, observed: `grep -c " $d[0-9][0-9] "` for `count_class` returns `70` for 4xx and `55` for 5xx because it also matches byte counts such as `420` and `512` — the grader catches it (`expected [50], got [70]`), and it is the moment to say why the course keys on *field 9* rather than on a pattern anywhere in the line.

Part 1: Prepare the exercise folder

Step 1: Go to the exercise and run the grader once.

a. From the course folder:

```
cd labs/lab5-text-processing/exercise
bash grade.sh
```

b. The first run copies `starter/reports.sh` into `work/` and grades that copy. Read the last line.

Expected: `---- lab5: 0 passed, 5 failed ----`. Each `FAIL` line names a check and shows `expected [...], got [...]`. That list is your to-do list; the checks run top to bottom, so fix them in order.

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.

Step 2: Open your working copy.

a. Open `work/reports.sh` in your editor. It contains two functions, `top_ips` and `count_class`. Each `# TODO` comment quotes the shape of the line you need. The file is *sourced* by the grader, so it only has to **define** the functions — it must not run anything at the top level.

Part 2: Build `top_ips`

`top_ips FILE N` must print the N most frequent client IPs, most frequent first, one per line, in the form `<ip> <count>`.

Step 1: Write the counting pipeline.

a. Replace the `:` placeholder in `top_ips` with the pipeline from S05.3, applied to field 1 and limited to N lines:

```
top_ips() {
  local f="$1" n="$2"
  awk '{print $1}' "$f" | sort | uniq -c | sort -rn | head -n "$n" | awk '{print $2,
$1}'
}
```

b. Read it left to right: `awk '{print $1}'` keeps only the IP of every line; `sort` puts equal IPs next to each other; `uniq -c` collapses each run of equal lines into one line prefixed by its count; `sort -rn` orders those lines by the count, largest first; `head -n "$n"` keeps the top N; the last `awk` swaps the two columns so the IP comes first.

Note: `sort` must come **before** `uniq -c`. `uniq` only compares each line with the line *before* it, so without the sort every IP would be counted as `1` again and again (observed in S05.3: six lines of `1`).

Step 2: Try it by hand.

a. Source your file and call the function on the fixture log the grader generates:

```
tmp=$(mktemp -d) && bash ../../../seed/make_logs.sh "$tmp" >/dev/null
source work/reports.sh
top_ips "$tmp/access.log" 3
```

Expected: three lines, the first being `10.0.0.1 50`.

Part 3: Build `count_class`

`count_class FILE D` must print how many lines have a status code (field 9) that starts with the digit D — `count_class access.log 4` is the number of 4xx responses.

Step 1: Write the awk filter.

a. Replace the `printf '0\n'` placeholder:

```
count_class() {
  local f="$1" d="$2"
  awk -v d="$d" '$9 ~ ("^" d) {c++} END{print c+0}' "$f"
}
```

b. How it works: `-v d="$d"` hands the shell value to awk **as data**, under the awk variable `d`; `$9 ~ ("^" d)` is true when field 9 starts with that digit; `{c++}` counts the matching lines; `END{print c+0}` prints the

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.

total once, after the last line — the `+0` turns an unset `c` into `0` when nothing matched, instead of an empty line.

Note: Do **not** splice the shell variable into the awk program (`'$9 ~ /^'"$d"'/'`). It passes this grader but it builds awk *code* out of data: a value containing `/`, `}` or a quote would change the program. Reflection question 2 is about exactly this.

Step 2: Try it by hand.

```
count_class "$tmp/access.log" 4
count_class "$tmp/access.log" 5
count_class "$tmp/access.log" 2
rm -rf "$tmp"
```

Expected: `50`, `5`, `85`.

Part 4: Verify with the grader

Step 1: Run the grader until it is green.

```
bash grade.sh
```

Expected: `---- lab5: 5 passed, 0 failed ----`. The five checks are: `top IP is 10.0.0.1 with 50`; `top_ips N=3 returns 3 lines`; `4xx count is 50`; `5xx count is 5`; `2xx count is 85`.

b. If a check still fails, read only the **first** `FAIL` line and compare its `expected` and `got` values with the step above. Three common causes:

Symptom	Cause	Fix
`got [50 10.0.0.1]`	Columns are in `uniq -c` order	Add the final `awk '{print $2, $1}'`
`got [70]` for 4xx	A pattern matched byte counts too	Key on field 9 with `awk`, not on a pattern anywhere in the line
An empty line instead of `0`	`print c` without `+0`	`END{print c+0}`

Step 2: Reset if you want to start over.

```
rm -rf work && bash grade.sh
```

Part 5: Reflection

Create `work/REFLECTION.md` and answer both questions in your own words (3–5 sentences each). The answers are graded: they must name the **mechanism**, not just state a rule.

Question 1: Why must the pipeline `sort` before `uniq -c`? Say what `uniq` actually compares, and what the top-talkers report would show without the sort.

> **Answer:** **Met:** `uniq` compares each line only with the previous one, so duplicates must be adjacent; without `sort` every IP would show a count of 1 (or fragmented counts) and the "top talker" would be wrong. **Strong:** also notes that the `sort -rn` afterwards orders by the *count* produced by `uniq -c`, or quotes the observed six-1s output from S05.3. **Not met:** "uniq needs sorted input" with no reason.

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.

Question 2: `count_class` passes the class digit with `awk -v d="$d"`. Why not write the digit straight into the awk program (`'$9 ~ /^'"$d"'/'`)? Name the mechanism and one input that would break the spliced version.

> **Answer: Met:** `-v` passes the digit as a value; splicing the shell variable into the single-quoted program builds awk *code* from data, so a value containing awk syntax (quotes, `/`, braces) changes the program. **Strong:** gives a concrete breaking input (a `/` or a `}`) or names it as injection and links it to the course's "config is data" rule. **Not met:** "`-v` is the recommended way" / "quotes are tricky" with no mechanism.
>
> *Calibration.* Met: "The awk program is one single-quoted string; closing the quote to paste `$d` in means the digit becomes part of the program text. `-v d="$d"` keeps it a variable, so a `d` like `4/{print 1} #` would be a harmless pattern instead of new code." Not met: "Using -v is cleaner and avoids quoting problems." — style, no mechanism; return it with "what would awk do with a `}` inside `$d`?".

Hand-in and grading

Hand in `work/reports.sh`, `work/REFLECTION.md`, and your final grader output. **Pass** = the grader ends `0 failed` **and** both reflection answers name the mechanism. There is no limit on attempts.

> **Instructor Note: Grading procedure (≈5 min per learner).**
>
> 1. Grade their copy without touching it: `GRADE_TARGET=/path/to/their/work bash labs/lab5-text-processing/exercise/grade.sh` → require `---- lab5: 5 passed, 0 failed ----`.
> 2. Read `REFLECTION.md` against the descriptors above. Record `A-06 PASS` only with 5/5 **and** both reflections at Met or better. No partial credit.
> 3. **Acceptable variations:** `cut -d' ' -f1` instead of the first `awk` (passes on the fixture); `sort | uniq -c | sort -k1,1nr` (passes). The spliced awk program `'$9 ~ /^'"$d"'/'` passes the grader but **fails Question 2** — coach it out. A `count_class` without `+0` passes the grader (no class is empty in the fixture) but prints an empty line for an empty class — mention it in feedback.
> 4. **Model feedback (Not yet):** "`top IP is 10.0.0.1 with 50` got `50 10.0.0.1`: the columns are in `uniq -c` order; add the final `awk '{print $2, $1}'`." Remediation: S05.3.
> 5. **Equivalent reassessment:** same grader; Q1′ "Why does `top_ips` use `head -n "$n"` after the second sort rather than before?" (Met: the limit must apply to the *ranked* list) and Q2′ "Why `END{print c+0}` and not `END{print c}`?" (Met: an unset awk variable prints as empty; `+0` forces `0`).

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.

Instructor Note: Appendix — reference solution (`solution/reports.sh`, observed 5/5):

```
#!/usr/bin/env bash
# Lab 5 solution — the counting pipeline and an awk filter.

# top_ips: the N most frequent client IPs (field 1), "<ip> <count>" most first.
top_ips() {
  local f="$1" n="$2"
  awk '{print $1}' "$f" | sort | uniq -c | sort -rn | head -n "$n" | awk '{print $2,
$1}'
}

# count_class: how many lines have a status (field 9) starting with <digit>.
count_class() {
  local f="$1" d="$2"
  awk -v d="$d" '$9 ~ ("^" d) {c++} END{print c+0}' "$f"
}
```

© 2026 SecMLOps · Course 0 — Bash Scripting. Course material; not for redistribution outside the course.